

R5.09 Virtualisation avancée

Michaël Hauspie

BUT 3

Plan du cours

1 Semaine 3

- Terraform/OpenTofu
- Configuration d'une infrastructure, illustration avec Openstack

Terraform/OpenTofu

- Outil écrit en Go
- Se présente sous la forme d'un binaire unique
- Se *contente* d'appeler les API par l'intermédiaire d'un ou plusieurs *providers* (OpenStack, Proxmox, DigitalOcean, Azure, AWS, GCP, ...)

Description d'infrastructure

- Se présente sous la forme d'un ou plusieurs fichiers texte (extension .tf)
- Ces fichiers peuvent être intégrés au contrôle de version
- Terraform traduit ces fichiers en une série d'appel aux providers
 - ▶ dépend de l'état actuel
 - ▶ et de l'état désiré

Example

```
resource "openstack_compute_instance_v2" "basic" {  
  name           = "basic"  
  image_name     = "debian13"  
  flavor_name    = "normale"  
}
```

Initialisation

La première étape est de définir les *providers* à utiliser dans un fichier `.tf`.
Par exemple, `main.tf`

```
terraform {  
  required_providers {  
    openstack = {  
      source = "terraform-provider-openstack/openstack"  
    }  
  }  
}
```

Et à initialiser terraform dans le répertoire courant

```
tofu init
```

Initialisation

La commande `tofu init` initialise un répertoire

Première commande à lancer pour créer les fichiers qui ne seront pas sous contrôle de version

- état local et distant
- modules externes
- *providers*
- ...

Contenu d'un fichier terraform

On y trouve :

- des ressources : des éléments de l'infrastructure qui peuvent être :
 - ▶ une instance de machine virtuelle
 - ▶ un réseau virtuel
 - ▶ un enregistrement DNS
 - ▶ ...
- des sources de données
 - ▶ généralement fournies par le *provider*
 - ▶ permettent d'obtenir des informations diverses (type d'instances possibles sous openstack par exemple)
 - ▶ et de les utiliser dans les ressources
- des variables
 - ▶ en entrée
 - ▶ en sortie : récupérer les adresses IP des instances créées par exemple

La commande tofu

La commande donne accès à un ensemble de sous-commandes :

- `init` : initialise le répertoire courant
- `plan` : planifie le déploiement : affiche ce qui *serait* fait lors de la mise en œuvre du déploiement
- `apply` : effectue le déploiement
 - ▶ crée et/ou met à jour les ressources
- `destroy` : supprime le déploiement
 - ▶ supprime toutes les ressources
- `refresh` : mets à jour l'état local en fonction de l'état distant
 - ▶ utile pour les variables de sortie par exemple

Plan du cours

1 Semaine 3

- Terraform/OpenTofu
- Configuration d'une infrastructure, illustration avec Openstack

Documentation

Pour l'utilisation d'un provider, il faut se référer à sa documentation

- configuration du provider
- ressources disponibles
- sources de données disponibles
- ...

Pour openstack :

- <https://registry.terraform.io/providers/terraform-provider-openstack/openstack/latest/docs>
- <https://search.opentofu.org/provider/terraform-provider-openstack/openstack/latest>

Définition du provider

```
terraform {  
  required_providers {  
    openstack = {  
      source = "terraform-provider-openstack/openstack"  
    }  
  }  
}
```

Configuration du provider

```
provider "openstack" {  
    user_name     = "michael.hauspie"  
    password      = "....."  
    auth_url      = "http://cloud.univ-lille.fr:5000/v3/"  
    /* ... */  
}
```

Dans le cas d'openstack¹, tout peut se faire à l'aide de variables d'environnement, *i.e* on n'a pas besoin de définir ce bloc

- Plus simple de changer de fournisseur openstack
- On est sûr de ne pas mettre de secret dans le git...

1. c'est vrai pour la plupart des *providers*

Variables d'environnement Openstack 1/2

- Script téléchargeable directement depuis l'interface d'openstack

openstack. michael.hauspie

Projet

Accès API

Projet / Accès API

Accès API

Affichage de 5 éléments

Service	Point d'accès au service
Compute	http://cloud.univ-lille.fr:8774/v2.1
Identity	http://cloud.univ-lille.fr:5000/v3/
Image	http://cloud.univ-lille.fr:9292
Network	http://cloud.univ-lille.fr:9696
Placement	http://cloud.univ-lille.fr:8778

Affichage de 5 éléments

Voir les données d'identification Télécharger le fichier RC d'OpenStack

Variables d'environnement Openstack 2/2

```
# To use an OpenStack cloud you need to authenticate against the Identity
# service named keystone, which returns a **Token** and **Service Catalog**.
# The catalog contains the endpoints for all services the user/tenant has
# access to - such as Compute, Image Service, Identity, Object Storage, Block
# Storage, and Networking (code-named nova, glance, keystone, swift,
# cinder, and neutron).
#
# *NOTE*: Using the 3 *Identity API* does not necessarily mean any other
# OpenStack API is version 3. For example, your cloud provider may implement
# Image API v1.1, Block Storage API v2, and Compute API v2.0. OS_AUTH_URL is
# only for the Identity API served through keystone.
export OS_AUTH_URL=http://cloud.univ-lille.fr:5000/v3/
# With the addition of Keystone we have standardized on the term **project**
# as the entity that owns the resources.
export OS_PROJECT_ID=...
export OS_PROJECT_NAME="michael.hauspie"
export OS_USER_DOMAIN_NAME="Default"
if [ -z "$OS_USER_DOMAIN_NAME" ]; then unset OS_USER_DOMAIN_NAME; fi
export OS_PROJECT_DOMAIN_ID="default"
if [ -z "$OS_PROJECT_DOMAIN_ID" ]; then unset OS_PROJECT_DOMAIN_ID; fi
# unset v2.0 items in case set
unset OS_TENANT_ID
unset OS_TENANT_NAME
# In addition to the owning entity (tenant), OpenStack stores the entity
# performing the action as the **user**.
export OS_USERNAME="michael.hauspie"
# With Keystone you pass the keystone password.
echo "Please enter your OpenStack Password for project $OS_PROJECT_NAME as user $OS_USERNAME: "
read -sr OS_PASSWORD_INPUT
export OS_PASSWORD=$OS_PASSWORD_INPUT
# If your configuration has multiple regions, we set that information here.
# OS_REGION_NAME is optional and only valid in certain environments.
export OS_REGION_NAME="RegionOne"
# Don't leave a blank variable, unset it if it was empty
if [ -z "$OS_REGION_NAME" ]; then unset OS_REGION_NAME; fi
export OS_INTERFACE=public
export OS_IDENTITY_API_VERSION=3
```

Création d'une instance

Pour créer une instance sur openstack il faut, à minima :

- son nom
- le modèle d'image à utiliser
- le gabarit à utiliser (*flavor*) : la *taille* de l'instance

La ressource à utiliser est openstack_compute_instance_v2

```
resource "openstack_compute_instance_v2" "basic" {  
  name          = "basic"  
  image_name     = "debian13"  
  flavor_name    = "normale"  
}
```

Utilisation des variables de sortie

Terraform peut nous renseigner sur l'état du déploiement à l'aide d'une variable de sortie.

Par exemple, obtenir l'adresse IP de l'instance créée

```
output "ip" {  
    value = openstack_compute_instance_v2.basic.access_ip_v4  
}
```

Ici, c'est la source de données `openstack_compute_instance_v2` qui est utilisée.

Plusieurs objet similaires : `count`

Par défaut, un bloc de ressource correspond à un objet de l'infrastructure.
On peut souhaiter créer plusieurs objets similaires

```
resource "openstack_compute_instance_v2" "basic" {  
    count          = 4  
    name           = "basic-${count.index}"  
    image_name     = "debian13"  
    flavor_name    = "normale"  
}
```

Utilisation de variables d'entrées avec : `for_each` 1/2

`for_each` permet de parcourir des variables (listes, dictionnaires)

```
variable "projet" {
  description = "La configuration de notre infra"
  type = map
  # Chaque élément de notre map est une VM
  # avec un gabarit différent
  default = {
    vm1 = {
      flavor = "normale",
      hostname = "vm1",
    },
    vm2 = {
      flavor = "petite"
      hostname = "vm2",
    },
    vm3 = {
      flavor = "petite",
      hostname = "vm3",
    },
  },
}
```

Utilisation de variables d'entrées avec : `for_each` 1/2

On utilise ensuite `for_each` dans le bloc ressource

```
resource "openstack_compute_instance_v2" "basic" {  
  for_each      = var.projet  
  
  name          = "${each.value.hostname}"  
  image_name     = "debian13"  
  flavor_name    = "${each.value.flavor}"  
}  
  
output "ip" {  
  value = {  
    for k, v in openstack_compute_instance_v2.basic : k => v.access_ip_v4  
  }  
}
```

Utilisation de sources de données

On peut effectuer des requêtes au *provider* à l'aide de sources de données

```
data "openstack_compute_flavor_v2" "small" {  
  vcpus = 1  
  ram   = 1024  
}
```

Si un gabarit correspondant existe, on peut utiliser le nom dans la ressource

```
resource "openstack_compute_instance_v2" "basic" {  
  name           = "basic"  
  image_name     = "debian13"  
  flavor_name    = data.openstack_compute_flavor_v2.small.name  
}
```

Ou dans les variables de sortie

```
output "gabarit" {  
  value = data.openstack_compute_flavor_v2.small.name  
}
```